Experiment 3:

Middleware – Logging & Error Handling

1. Aim :

To implement custom middleware in Express.js for request logging and global error handling.

2. Requirements

- Node.js (v18+)
- VS Code + Thunder Client / Postman
- Express.js

3. Theory :

Middleware in Express.js are functions that have access to the request and response objects. They can execute code, modify the request/response, and call the next middleware in the stack. Request logging middleware helps in monitoring and debugging by logging every incoming request. Error handling middleware is a special middleware with four parameters (err, req, res, next) used to catch and handle errors gracefully across the application.

4. Algorithm :

Step 1: Create project folder and initialize Node.js project.

Step 2: Install Express.js.

Step 3: Create middleware folder and logging middleware.

Step 4: Create error handling middleware.

Step 5: Set up main server file with routes and middleware.

Step 6: Run the server.

Step 7: Test using Thunder Client / Postman.

5. Procedure

Step 1: Create Project Folder

Bash

mkdir exp3-middleware

cd exp3-middleware

npm init -y

npm install express

Step 2: Create middleware folder

Bash

mkdir middleware

Step 3: Create middleware/logger.js

Step 4: Create middleware/errorHandler.js

Step 5: Create server.js

Step 6: Run the server Bash

node server.js

Step 7: Test the routes using Thunder Client or Postman.

6. Program

middleware/logger.js

JavaScript

```
function logger(req, res, next) {
    const timestamp = new Date().toISOString();
    console.log(`[${timestamp}] ${req.method} ${req.url}`);
    next(); // IMPORTANT: must call next()
}

module.exports = logger;
```

middleware/errorHandler.js

JavaScript

```
function errorHandler(err, req, res, next) {
    console.error('Error:', err.message);
    res.status(err.status || 500).json({
        error: err.message || 'Internal Server Error'
    });
}

module.exports = errorHandler;
```

server.js

JavaScript

```
const express = require('express');
const app = express();
const PORT = 3000;

const logger = require('./middleware/logger');
const errorHandler = require('./middleware/errorHandler');

app.use(express.json());
app.use(logger);   // Apply logger to ALL routes

// Normal route
app.get('/data', (req, res) => {
    res.json({ info: 'Here is your data' });
});

// Route that triggers error
app.get('/error', (req, res, next) => {
    const err = new Error('Something went wrong!');
    err.status = 400;
    next(err);
});

// Error handler must be placed AFTER all routes
app.use(errorHandler);
```

```
app.listen(PORT, () => {
    console.log(`Server running on http://localhost:${PORT}`);
});
```

7. Sample Input

- **GET Request:** http://localhost:3000/data
- **GET Request (Error):** http://localhost:3000/error

8. Output

Terminal Log Example:

text

```
[2026-06-16T23:30:00.000Z] GET /data
[2026-06-16T23:31:15.000Z] GET /error
```

GET /data Response:

JSON

```
{
  "info": "Here is your data"
}
```

GET /error Response:

JSON

```
{
  "error": "Something went wrong!"
}
```

9. Result Thus, custom middleware for request logging and error handling was successfully implemented and tested in a Node.js Express application.